

# DistTrain: Addressing Model and Data Heterogeneity with Disaggregated Training for Multimodal Large Language Models

Zili Zhang\* Yinmin Zhong\* Yimin Jiang\* Hanpeng Hu†

Jianjian Sun† Zheng Ge† Yibo Zhu† Daxin Jiang† Xin Jin\*

\*School of Computer Science, Peking University †StepFun \*Independent Researcher

## ABSTRACT

Multimodal large language models (LLMs) empower LLMs to ingest inputs and generate outputs in multiple forms, such as text, image, and audio. However, the integration of multiple modalities introduces heterogeneity in both the model and training data, creating unique systems challenges.

We propose DistTrain, a disaggregated training system for multimodal LLMs. DistTrain incorporates two novel disaggregation techniques to address model and data heterogeneity, respectively. The first is *disaggregated model orchestration*, which separates the training for modality encoder, LLM backbone, and modality generator. This allows the three components to adaptively and independently orchestrate their resources and parallelism configurations. The second is *disaggregated data preprocessing*, which decouples data preprocessing from training. This eliminates resource contention between preprocessing and training, and enables efficient data reordering to mitigate stragglers within and between micro-batches caused by data heterogeneity. We evaluate DistTrain across different sizes of multimodal LLMs on a large-scale production cluster. The experimental results show that DistTrain achieves 54.7% Model FLOPs Utilization (MFU) when training a 72B multimodal LLM on 1172 GPUs and outperforms Megatron-LM by up to 2.2× on training throughput.

## CCS CONCEPTS

• Computer systems organization → Cloud computing; • Computing methodologies → Machine learning.

## KEYWORDS

Large Language Models, Multimodal Models, Distributed Training

## ACM Reference Format:

Zili Zhang, Yinmin Zhong, Yimin Jiang, Hanpeng Hu, Jianjian Sun, Zheng Ge, Yibo Zhu, Daxin Jiang, Xin Jin. 2025. DistTrain: Addressing Model and Data Heterogeneity with Disaggregated Training for Multimodal Large

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

SIGCOMM '25, September 8–11, 2025, Coimbra, Portugal

© 2025 Copyright held by the owner/author(s). Publication rights licensed to the Association for Computing Machinery.

ACM ISBN 979-8-4007-1524-2/25/09.

<https://doi.org/10.1145/3718958.3750472>

Language Models. In *SIGCOMM '25, August 8–August 11, 2025, Coimbra, Portugal*, 15 pages.

## 1 INTRODUCTION

Traditional multimodal models, which focus solely on either cross-modal feature representation (e.g., CLIP [48]) or multimodal content generation (e.g., Stable-Diffusion [53] and VAE [36]), cannot simultaneously process multimodal inputs and generate multimodal outputs. This restricts their capabilities and hinders seamless human interaction. Multimodal large language models (LLMs) emerge to integrate the strengths of LLMs [18, 62, 63] with traditional multimodal models to simultaneously ingest multimodal inputs and generate multimodal outputs in auto-regressive manner. Multimodal LLMs have already shown great potential in tasks such as vision understanding and generation [25, 37, 42, 68, 76], audio comprehension [16, 54], and embodied AI [27, 74]. Many organizations are actively developing multimodal LLMs, such as OpenAI's GPT-4o [5], Google's Gemini [6], and Meta's Llama3 [7], etc.

Figure 1 depicts the model architecture of multimodal LLMs, comprising three modules: modality encoder, LLM backbone, and modality generator [25, 70, 75]. We emphasize this architecture underpins most state-of-the-art multimodal LLMs (Table 1). These modules are linked by projectors, which may incorporate MLP or cross-attention. The modality encoder transforms inputs from various modalities into different modality tokens. These tokens from different modality are interleaved as a *sequence* for LLM training. The modality generator translates the processed information back into coherent outputs tailored to each modality.

Training multimodal LLMs is more challenging than training traditional multimodal models. Traditional multimodal model training frameworks like DistMM [32] lack support for simultaneous training of encoders, LLMs, and generators, particularly techniques tailored for LLMs. The current practice for training multimodal LLMs [13, 69] extends LLM training frameworks (e.g., Megatron-LM [57]) with additional modality encoder and generator modules, which uses a monolithic approach. In this context, the monolithic training refers to two aspects. First, it uses the same data and tensor parallelism strategies of distributed training across the modality encoder, LLM backbone, and modality generator modules. As for pipeline parallelism, the encoder and generator modules are incorporated as additional pipeline stages in the training pipeline. Second, it couples data preprocessing into training process, and co-locates data preprocessing and training on the same machines.

The monolithic approach falls short of handling unique systems challenges in multimodal LLM training. Compared to traditional

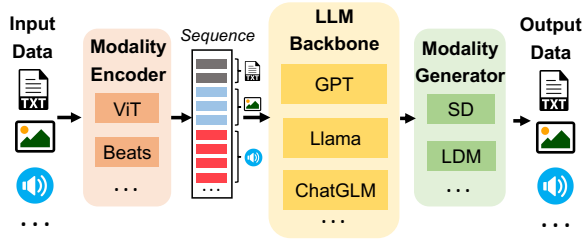


Figure 1: The architecture of multimodal LLMs.

LLM training that only handles text, the multiple modalities in multimodal LLM training introduces *model heterogeneity* and *data heterogeneity*. Model heterogeneity stems from processing multiple modalities with different modules (i.e., encoder, LLM backbone, and generator) that vary dramatically in size and operator complexity. Using the same parallelism strategies across different modules introduces severe pipeline bubbles, resulting in poor GPU utilization (§2.2). Data heterogeneity emerges from processing multimodal data that differ significantly in volume and processing cost, e.g., a high-resolution image is much larger and requires more resources for preprocessing than a line of text. In the monolithic approach, where data preprocessing and training are integrated and co-located, the substantial processing cost significantly hinders the training process. Additionally, the heterogeneity across different modality data (i.e., each training sample has a varying number of modality tokens) leads to inter-microbatch and intra-microbatch training stragglers, which prolong the training duration and exacerbate the pipeline bubbles (§2.3). In our initial efforts using the monolithic approach, we observed extremely low MFU in production training, as low as 20% (§7.1).

To this end, we propose DistTrain, a disaggregated training system for multimodal LLMs. DistTrain applies the systems principle of disaggregation in the context of multimodal LLM training to address the unique systems challenges. We design two novel disaggregation techniques, which are *disaggregated model orchestration* and *disaggregated data preprocessing*, to address model and data heterogeneity, respectively.

For model heterogeneity, we analyze the pipeline bubbles caused by multiple modalities across different modules in multimodal LLMs, and identify their root causes (§2.2). To address this, we design *disaggregated model orchestration* to separate the training of modality encoder, LLM backbone, and modality generator. This separation enables adaptive orchestration of resources and parallelism configurations across the three modules. Building on this, we formulate the model orchestration as an optimization problem aimed at minimizing the training time per iteration. We then design an adaptive model orchestration algorithm that navigates the complicated search space to find the *optimal* resource and parallelism configurations. Disaggregated model orchestration is able to minimize pipeline bubbles caused by model heterogeneity and achieves *optimal* training efficiency.

For data heterogeneity, we characterize the training data of different modalities, and unveil their preprocessing overheads and the stragglers they brought into the training process (§2.3). We categorize the stragglers into *intra-microbatch* and *inter-microbatch* stragglers. We design *disaggregated data preprocessing* to eliminate the interference between preprocessing and training. Importantly,

| Multimodal LLM | Encoder(s)                    | LLM Backbone | Generator(s)                  |
|----------------|-------------------------------|--------------|-------------------------------|
| Flamingo [13]  | NFNet [17]                    | GPT-3 [18]   | LM-Head                       |
| LLaVA [41]     | CLIP [48]                     | Vicuna [2]   | LM-Head                       |
| PaLM-E [27]    | ViT [26]                      | PaLM [23]    | LM-Head                       |
| EMU [60]       | EVA-CLIP [59]                 | Llama [62]   | LM-Head, SD [53]              |
| Bagel [24]     | ViT [59]                      | Qwen2.5 [10] | LM-Head, VAE [36]             |
| VideoPoet [37] | MAGViT [71], SoundStream [72] | GPT [49]     | MAGViT [71], SoundStream [72] |

Table 1: Some examples of multimodal LLM's architecture.

such disaggregation enables flexible data preprocessing, and provides new opportunities for *data reordering* to mitigate stragglers without additional overhead. We use intra-microbatch reordering to evenly distribute the load across data parallelism groups, and inter-microbatch reordering to hide the training stragglers into the training pipeline. This dual-level reordering effectively mitigates the stragglers caused by data heterogeneity. Importantly, since the reordering only permutes the sequence of commutative gradient accumulation operations, it preserves the training's convergence semantics.

In summary, we make the following contributions.

- We identify the unique system challenges introduced by multiple modalities in multimodal LLM training, and summarize them as model and data heterogeneity.
- We propose DistTrain, a disaggregated training system for multimodal LLMs, and design two novel techniques—disaggregated model orchestration and disaggregated data preprocessing—to address model and data heterogeneity.
- We implement DistTrain and conduct experiments on our production cluster with thousands of GPUs. The experimental results show that DistTrain achieves 54.7% MFU when training a 72B multimodal LLM on 1172 GPUs and outperforms Megatron-LM by up to 2.2× on training throughput.

## 2 MOTIVATION

### 2.1 Multimodal LLM Training

**Traditional multimodal models.** Traditional multimodal models encompass diverse architectures, such as contrastive learning [48] and stable diffusion [53] (SD). Contrastive learning uses encoder models to capture cross-modal similarities, primarily for feature representation. Stable diffusion generates images from text prompts by iteratively refining random noise through a denoising process, enabling text-to-image generation. However, these models are task-specific and lack the flexibility to handle diverse multimodal scenarios, which cannot process multimodal inputs and generate multimodal outputs simultaneously, restricting their model capabilities in complex real-world settings and hindering seamless interaction with humans.

**Multimodal LLMs.** Large language models (LLMs)[6, 7, 12], built on homogeneous transformer layers and trained on extensive text corpora, excel at understanding and generating high-quality text but are limited to text-only processing. Multimodal LLMs address these limitations by integrating the strengths of LLMs with traditional multimodal models, enabling the simultaneous processing

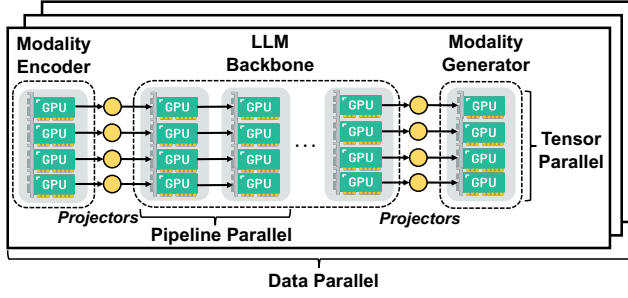


Figure 2: Training multimodal LLMs with Megatron-LM.

of diverse inputs (e.g., images, audio, video) and generating multimodal outputs in auto-regressive manner. For example, GPT-4o [5] gains wide attention by facilitating more natural interaction with humans through both visual and auditory modalities.

Figure 1 illustrates the model architecture of a multimodal LLM, which consists of three modules: a modality encoder, an LLM backbone, and a modality generator [13, 70, 75]. Different from traditional multimodal models, this generative model facilitates a more robust and interpretable process for understanding and generating multimodal contents through auto-regressive decoding. Specifically, the modality encoder transforms input data from different modalities (e.g., ViT [26] for images and Beats [21] for audios) into an intermediate representation, which is then converted into different modality tokens with input projector (e.g., MLP and cross-attention). These tokens from different modalities form a *sequence* for LLM training. The LLM backbone, typically a transformer model (e.g., GPT [18, 50] and Llama [7]), processes the input *sequence* to extract intricate data patterns and inter-modal relationships. The LLM backbone’s output is refined by an output projector and then passed to a modality generator (e.g., Diffusion [53] for images, AudioLDM [40] for audio), which converts the processed information into the corresponding modal outputs. Table 1 summarizes the three modules of some state-of-the-art multimodal LLMs. We omit text tokenizer (e.g., BPE) and encoder (e.g., text embedding) from the table’s encoder column. LM-Head is responsible for decoding the text tokens

Multimodal LLM training necessitates training all three modules simultaneously. Additionally, during different training phases, specific modules are frozen to stabilize training loss [22, 24]. Regarding the data, input sequences comprise text, image, and audio tokens. The data from different modalities are encoded into *subsequences* which are then interleaved to form fixed-length training *sequences* [61].

**Training frameworks.** Multimodal model training frameworks such as DistMM [32] do not support training encoder, LLM backbone and generator simultaneously. Especially, they lack support training techniques [20, 46, 51] for LLMs. In contrast, LLM training frameworks like Megatron-LM [57] can be extended with modality encoder and generator modules, making them widely used in production multimodal LLM training, e.g., Flamingo [13], DeepSeek-VL2 [69] and VideoPoet [37]. Figure 2 shows training multimodal LLMs with Megatron-LM. Megatron-LM adds multimodal modules as additional layers, and incorporates additional pipeline parallelism (PP) stages to accommodate encoder and generator. The

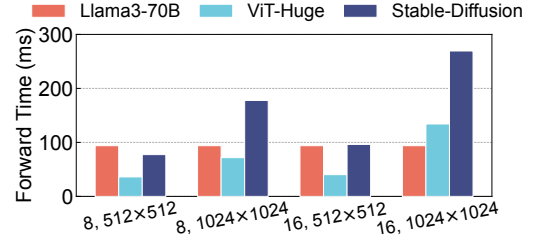


Figure 3: Forward time under different input configurations.

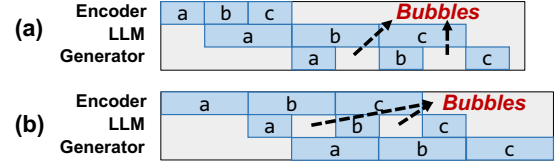


Figure 4: Two types of pipeline bubbles.

same tensor parallelism (TP) strategy used in the LLM backbone is applied to encoder and generator. If the encoder and generator are not large enough, they are replicated across the GPUs in the TP group to maximize resource utilization. As for data parallelism (DP), Megatron-LM applies the same DP strategy to the multimodal modules as the LLM backbone. The projector is co-located with the encoder and generator, and is replicated across the GPUs in the TP group. Moreover, data preprocessing is incorporated into the training process and co-located with training on the same node. This approach is *monolithic* in terms that the encoder and generator use the same parallelism strategy as the LLM backbone, and that preprocessing and training are integrated and co-located. Such a monolithic approach introduces significant computation imbalance stemming from *model* and *data* heterogeneity.

## 2.2 Model Heterogeneity

**Characterization.** Each module in multimodal LLMs has different computational demands due to varying operators and inputs. For instance, ViT, as modality encoder, is constructed with narrow transformer layers (i.e., small hidden size), whereas the LLM backbone is built with wide transformer layers (i.e., large hidden size). Meanwhile, Diffusion, as modality generator, utilizes a combination of convolution and attention layers. The diversity in model architecture results in distinct computation time for each module. Figure 3 shows varying forward time under different input configurations with Megatron-LM. We demonstrate one PP stage of LLM backbone with PP size of 10 and TP size of 8. The first configuration parameter is the number of images in the 8K input sequence, and the second is the image resolution. The results reveal a key performance disparity: while the LLM backbone maintains a consistent forward time, the modality encoder and generator exhibit significant variations.

**Pipeline imbalance.** The computation imbalance between modules leads to two types of bubbles in pipeline parallelism. The first arises in the modality encoder and generator stages, as shown in Figure 4(a), resulting from their inadequate utilization of allocated GPU resources. The second type arises in the stages of the LLM backbone, as shown in Figure 4(b). This is because the intensive

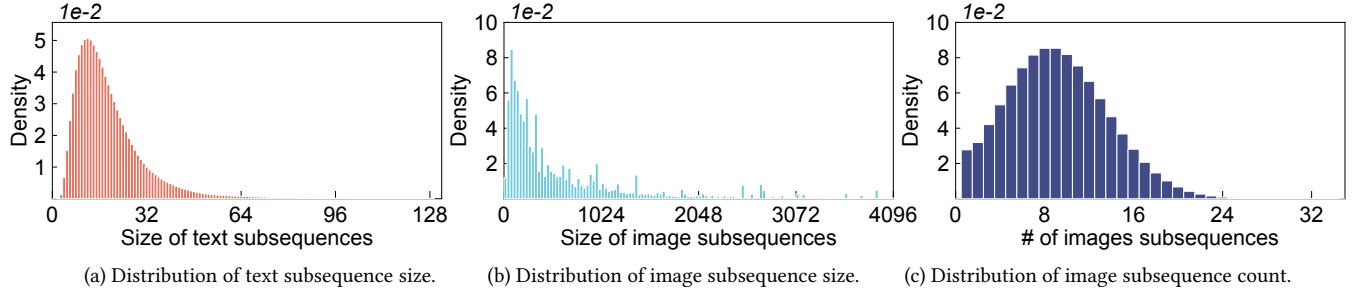


Figure 5: Data heterogeneity in multimodal LLM training.

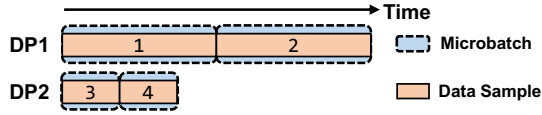


Figure 6: Intra-microbatch straggler (among DP groups).

computation demands of the encoder and generator increase their stage durations. Due to the pipeline dependency, the LLM stages are forced to wait for the multimodal stage to complete, thereby creating pipeline bubbles. The latter problem is particularly pronounced during large-scale multimodal LLM training, where the bulk of GPU resources are allocated to the LLM backbone. These pipeline bubbles, stemming from model heterogeneity, reduce the MFU significantly during training.

### 2.3 Data Heterogeneity

**Characterization.** The training data samples of Multimodal LLMs often combine lightweight text with heavyweight multimodal data. The latter significantly increases data preprocessing time. For example, a typical training sample could include a 256-word text sequence and ten  $1024 \times 1024$  RGB images. The text is just kilobytes, whereas the images are total of 120 megabytes. Preprocessing (e.g., decompression, resizing, and reordering) such samples can take several seconds, and prolong the co-located training process. Additionally, each input sequence consists of interleaved modality subsequences that exhibit highly skewed distributions. Focusing on images and texts, we perform data characterization on the LAION-400M dataset [55]. Each image (i.e., one image subsequence) is segmented into  $16 \times 16$  patches, and each patch is converted into one image token. The texts are tokenized through Llama tokenizer. The image tokens are interleaved with text tokens to create an 8K-token training sequence. As shown in Figure 5(a) and Figure 5(b), the sizes of text and image subsequences display highly skewed distributions. We further analyze the count of modality subsequence per training sample using image as an example. The count of image subsequences per training sample, shown in Figure 5(c), also shows a skewed distribution. Different sample sizes (i.e., modality tokens per sample) lead to varying computation time in the modality encoder and generator.

**Stragglers.** Data heterogeneity results in *intra-microbatch* and *inter-microbatch* stragglers within the PP stages of the modality encoder and generator. These stragglers exacerbate the computation imbalances and further reduce GPU utilization. It is noted that all

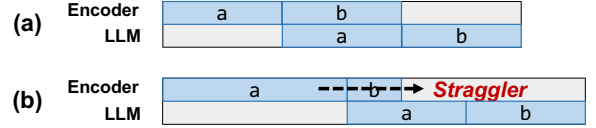


Figure 7: Inter-microbatch straggler.

microbatches within the LLM have the same computation time since the sequence length is fixed. We do not consider data heterogeneity between global batches, as each global batch contains numerous randomly shuffled training samples (e.g., thousands with a large DP size), which effectively smooths out data heterogeneity.

*Intra-microbatch straggler* arises since different DP groups handle variably-sized training samples. Illustrated in Figure 6, the first DP group (DP1) processes two large training samples within two microbatches. In contrast, the second DP group (DP2) processes two smaller samples in the same microbatches, completing them more quickly. Consequently, DP1 lags behind DP2 and becomes the straggler, which delays the overall training process.

*Inter-microbatch straggler* emerges from pipeline imbalances between microbatches. As depicted in Figure 7, the first pipeline stage is the modality encoder followed by one LLM backbone stage. Figure 7(a) illustrates the pipeline without data heterogeneity, where the modality encoder processes each microbatch with the same amount of time. In contrast, Figure 7(b) depicts the pipeline with data heterogeneity, where the forward time of the modality encoder varies largely across microbatches. The straggler (i.e., the microbatch a) significantly delays the training process of the subsequent PP stages, leading to a large pipeline bubble.

## 3 DISTTRAIN OVERVIEW

We propose DistTrain, a disaggregated training system for multimodal LLMs. DistTrain addresses model heterogeneity by disaggregated model orchestration (§4) and handles data heterogeneity by disaggregated data preprocessing (§5). Figure 8 shows an overview of DistTrain.

**DistTrain manager.** Before training, DistTrain employs a training manager to determine the resource allocation and parallelism strategy for each module in multimodal LLMs. The training manager first gathers the model architecture and training configuration (e.g., global batch size) from the user and samples a subset of training data to analyze the data distribution. Utilizing the information, it runs a series of benchmarking training trials and constructs a performance



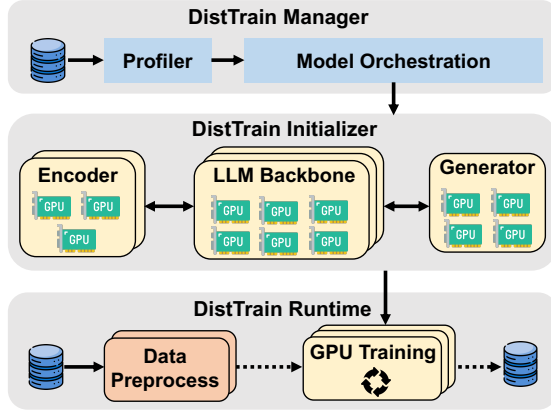


Figure 8: DistTrain overview.

profiler with linear interpolation to estimate each module’s computation and communication time. Based on the profiling results, the training manager decides the optimal resource allocation and parallelism strategy with *disaggregated model orchestration* for one specific training task, as detailed in §4.

**DistTrain initializer.** DistTrain then initializes the disaggregated model orchestration for modality encoder, LLM backbone, and modality generator, respectively. DistTrain allocates different numbers of GPUs and determines the parallelism strategy for each module. Each module then establishes the specific communication group. It then loads the model checkpoint from a distributed file system and shards the model parameters. Finally, DistTrain conducts several communication trials to warm up the system and test connectivity.

**DistTrain runtime.** At runtime, the dedicated CPU nodes (i.e., disaggregated data preprocessing nodes) retrieve training data samples from the distributed file system for preprocessing. It performs *data reordering* to reorder the training samples within one global batch while preserving the synchronous training semantics [1]. The reordering mitigates both intra-microbatch and inter-microbatch stragglers caused by data heterogeneity, as detailed in §5. In each iteration, the GPU training nodes receive the preprocessed data asynchronously from the CPU nodes. The data then undergoes sequentially through the modality encoder, LLM backbone, and modality generator in the training pipeline. Finally, the GPU training nodes synchronize the gradients and model parameters through collective communication. DistTrain employs ZERO-1 optimization [51] and mixed precision training [45] to reduce the memory footprint and accelerate the training of LLM backbone. Additionally, DistTrain adopts a dedicated process to periodically and asynchronously save model checkpoints to the distributed file system for fault tolerance.

## 4 ADDRESSING MODEL HETEROGENEITY

To address model heterogeneity, we first introduce *disaggregated model orchestration* to adaptively orchestrate the resource and parallelism configurations of the three modules in multimodal LLM training. We then formulate the problem to minimize the training

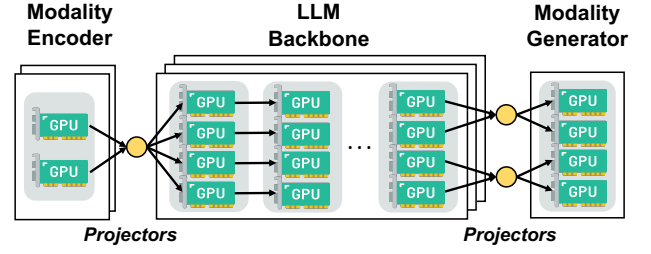


Figure 9: Disaggregated model orchestration in DistTrain.

time per iteration. Last, we present the adaptive model orchestration algorithm to find optimal resource and parallelism configurations.

### 4.1 Disaggregated Model Orchestration

Figure 9 illustrates the disaggregated model orchestration. Different from the monolithic model orchestration in Megatron-LM (i.e., Figure 2), DistTrain is able to adaptively adjust the resource allocation and parallelism strategy. For instance, DistTrain can allocate 4 GPUs (DP=2 and TP=2) to the modality encoder, 12 GPUs (DP=3 and TP=4) to the LLM backbone per PP stage, and 4 GPUs (DP=1 and TP=4) to the modality generator. Additionally, the projector layers are co-located with either the modality encoder or generator, with their number of replicas adapting as needed. We implement disaggregated model orchestration through a dedicated module, i.e., *parallelism unit*.

**Parallelism unit.** At training initialization, we need to establish the communication group according to the resource allocation and parallelism strategy. DistTrain introduces a module, parallelism unit, composed of one or more PP stages. Each unit can adopt its own DP and TP strategies and form a specific communication group. Inter-unit connections are facilitated by a *communication broker*, which bridges PP communication across parallelism units. Users are only required to specify the DP and TP configurations for each parallelism unit, and DistTrain automatically sets up the communication group and communication broker. DistTrain treats the modality encoder, LLM backbone, and modality generator as three individual parallelism units. For advanced workloads, DistTrain extends the capabilities of these units. To handle long sequences, it integrates sequence parallelism (SP) within the LLM backbone unit and automatically splits sequences within its SP group. Furthermore, DistTrain supports expert parallelism (EP) for the LLM backbone. Since EP and TP both perform parallel computation and communication within one layer, our subsequent formulation involving TP remains valid when TP is replaced with EP. The detailed implementation of parallelism unit is described in §6.

### 4.2 Problem Formulation

With disaggregated model orchestration, we are able to adaptively orchestrate the three modules. The problem now lies in determining the optimal resource allocation and parallelism strategy to minimize the training time per iteration. Exhaust search is infeasible due to the combinatorial search space, particularly for large clusters. One strawman solution is to allocate the resources proportional to the model flops of each module. However, this method falls short as it overlooks complex patterns in parallelism training. Different

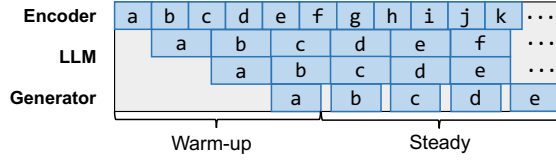


Figure 10: Multimodal LLM training pipeline.

DP and TP sizes in modality encoder could lead to different computation times while the model flops are the same. Before diving into our solution, we first formulate the optimization problem.

**LLM backbone.** We begin by formulating the LLM backbone, focusing on the forward pass as the backward pass mirrors it. In production LLM training, the microbatch size is set to a small number to prevent GPU memory overflow. DistTrain sets the microbatch size to a predefined constant,  $M$ . Assume the global batch size for one iteration as  $BS$  and the TP size of the LLM backbone as  $TP_{lm}$ . Let the PP and DP size of the LLM backbone be  $PP_{lm}$  and  $DP_{lm}$ . The LLM is divided into equal  $PP_{lm}$  PP stages, where each PP stage consists of some homogeneous transformer layers. The number of GPUs allocated to the LLM backbone is  $y = TP_{lm} \times DP_{lm} \times PP_{lm}$ . Let the forward time (including communication time) of the entire LLM for one sample be  $C_{lm}(TP_{lm})$ , where  $C_{lm}$  represents the forward time function. Therefore, the forward time of one PP stage for one microbatch with  $M$  samples is  $T_{lm} = \frac{C_{lm}(TP_{lm}) \times M}{PP_{lm}}$  since PP partitions LLM backbone's homogeneous layers sequentially. Besides, the number of microbatches per iteration is  $\frac{BS}{DP_{lm} \times M}$ .

**Modality encoder and generator.** In DistTrain, the modality encoder is regarded as a parallelism unit with PP size  $PP_{me}$ . Let the TP size be  $TP_{me}$  and the DP size be  $DP_{me}$ . The number of GPUs allocated to the modality encoder is  $x = TP_{me} \times DP_{me} \times PP_{me}$ . The microbatch size is  $\frac{DP_{lm} \times M}{DP_{me}}$  which is determined by the LLM backbone. Let the forward time of the entire modality encoder for one training sample be  $C_{me}(TP_{me})$ . The forward time of one PP stage for one microbatch in the modality encoder is  $T_{me} = \frac{DP_{lm} \times M}{DP_{me}} \times \frac{C_{me}(TP_{me})}{PP_{me}} = \frac{DP_{lm} \times TP_{me} \times M}{x} \times C_{me}(TP_{me})$ . Similarly, the forward time of one PP stage in the modality generator is  $T_{mg} = \frac{DP_{lm} \times TP_{mg} \times M}{z} \times C_{mg}(TP_{mg})$ , where  $z$  is the number of GPUs allocated to modality generator.

**Objective function.** Based on the preceding analysis, we next define the objective function for the optimization problem, i.e., the training time of one iteration. Figure 10 shows the pipeline of forward pass in multimodal LLM training. The LLM backbone comprises two PP stages, whereas the modality encoder and generator each consist of one PP stage. The forward pass is categorized into two phases: warm-up phase and steady phase. The warm-up phase spans from the initiation to the completion of the first microbatch to populate the pipeline. The duration of this phase is calculated as  $T_{warmup} = T_{lm} \times PP_{lm} + T_{me} \times PP_{me} + T_{mg} \times PP_{mg}$ , which is formulated as follows.

$$T_{warmup} = M \times C_{lm}(TP_{lm}) + \frac{DP_{lm} \times M}{DP_{me}} \times C_{me}(TP_{me}) + \frac{DP_{lm} \times M}{DP_{mg}} \times C_{mg}(TP_{mg}) \quad (1)$$

The duration of steady phase is dominated by the maximal computation time among PP stages, which is calculated as  $T_{steady} = \max(T_{lm}, T_{me}, T_{mg}) \times (\frac{BS}{DP_{lm} \times M} - 1)$ , where  $\frac{BS}{DP_{lm} \times M}$  is the number of microbatches per iteration. It is formulated as:

$$T_{steady} = \max \left\{ \begin{aligned} &\frac{DP_{lm} \times TP_{lm} \times M}{y} \times C_{lm}(TP_{lm}), \\ &\frac{DP_{lm} \times TP_{me} \times M}{x} \times C_{me}(TP_{me}), \\ &\frac{DP_{lm} \times TP_{mg} \times M}{z} \times C_{mg}(TP_{mg}) \end{aligned} \right\} \times \left( \frac{BS}{DP_{lm} \times M} - 1 \right) \quad (2)$$

Therefore, the objective function is to minimize  $T_{warmup} + T_{steady}$ . Considering backward pass with the similar computation pattern, the objective function is similar to that of the forward pass. Adjustments are made by changing  $C_{lm}$ ,  $C_{me}$ , and  $C_{mg}$  from forward time functions to the sum functions of forward and backward time. This formulation holds for GPipe [33] and 1F1B [29]. We will retrofit the formulation to adapt to VPP [46] later. TP communication is incorporated into the functions  $C_{lm}$ ,  $C_{me}$ , and  $C_{mg}$ , which are calibrated through interpolation from actual trials. The communication time of DP and PP is modeled as the communication volume divided by the bandwidth.

**Constraints.** Besides the objective function, we must consider constraints to ensure training feasibility. The first constraint is the resource constraint. The number of GPUs allocated to each module should be  $x + y + z \leq N$  where  $N$  is the total number of GPUs in the cluster and  $x, y, z$  are the number of GPUs allocated to each module in multimodal LLM. The second constraint involves GPU memory. We first consider LLM backbone. Memory allocation involves four parts: model parameters, gradients, optimizer states, and activation states. The memory of model parameters and gradients on one GPU is calculated as:  $\frac{P}{PP_{lm} \times TP_{lm}} = \frac{DP_{lm} \times P}{y}$ , where  $P$  denotes the total memory for the LLM parameters and gradients. The memory for optimizer states on one GPU (with ZeRO-1 optimization [51]) is:  $\frac{S}{y}$ , where  $S$  denotes the total memory for the optimizer states. ZeRO-1 partitions the optimizer states across DP groups. The peak memory for activation states on one GPU is:  $\frac{DP_{lm} \times L \times PP_{lm}}{y}$ , with  $L$  representing the memory needed for one microbatch (with microbatch size  $M$ ) of activation states across the entire LLM. In 1F1B, the first PP stage requires storage for  $PP_{lm}$  microbatches of activation states. We do not use GPipe in DistTrain since it consumes more memory without offering better training efficiency compared to 1F1B. The memory constraint ensures the sum of the four memory parts on one GPU does not exceed GPU capacity. As for the modality encoder and generator, the formulation is similar.

### 4.3 Adaptive Model Orchestration

The problem is non-convex, with  $x, y, z$ , and the DP, TP sizes as positive variables. Solving this with an exhaust search is impractical due to the large search space, particularly in large clusters. Designing an efficient algorithm that *adaptively* identifies the optimal resource allocation and parallelism strategy based on the training task poses a hard problem.

**Convex optimization.** Our key insight is to decompose the non-convex optimization problem into a series of simplified convex

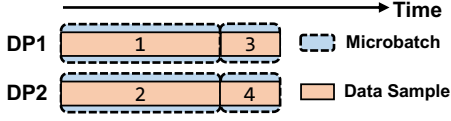


Figure 11: Intra-microbatch reordering.

problems with variables  $x, y, z$  (i.e., resource allocation). We confine the TP size to  $[1, 2, 4, 8]$  on an NVIDIA GPU node with 8 GPUs and adjust the DP size as a factor of  $BS$  to balance the computation across DP groups. If expert parallelism (EP) is used instead of TP, we confine EP size to factors of cluster size. The PP size of the LLM backbone is calculated as  $\frac{y}{DP_{lm} \times TP_{lm}}$ . The set of possible parallelism strategies is a manageable and finite set, i.e., the Cartesian product of TP and DP size. This allows us to enumerate all feasible TP and DP sizes, and transform the original optimization problem into a set of simplified problems. In the simplified problem, the objective function is maximal and additional of the functions:  $\frac{1}{x}$ ,  $\frac{1}{y}$  and  $\frac{1}{z}$ , where  $x, y, z$  are positive variables. Therefore, the objective function is convex. Similarly, the constraint functions are also convex. As a result, the simplified optimization problem is convex and can be efficiently solved to optimality by off-the-shelf solvers [30, 64]. The algorithm *adaptively* determines the optimal resource allocation and parallelism strategy.

Virtual pipeline parallelism [46] (i.e., interleaved 1F1B) reduces the warm-up time by dividing model into finer-grained virtual PP (VPP) stages. Each PP stage contains VPP-size VPP stages. In the warm-up phase, each PP stage launches the computation of one VPP stage, and the warm-up time is divided by VPP-size. To align our formulation with VPP, we proportionally reduce the warm-up time based on VPP-size.

## 5 ADDRESSING DATA HETEROGENEITY

To address data heterogeneity, we first introduce *disaggregated data preprocessing* that separates data preprocessing from training. This eliminates the interference between data processing and training and enables negligible overhead of data processing. We then introduce *disaggregated data reordering* to address data heterogeneity. We use intra-microbatch reordering to eliminate stragglers across DP groups, and inter-microbatch reordering to minimize pipeline bubbles and hide the training stragglers into the training pipeline as much as possible.

### 5.1 Disaggregated Data Preprocessing

As discussed in §2.3, preprocessing high-volume multimodal data (e.g., high resolution images) incurs substantial overhead. DistTrain disaggregates data preprocessing from training with a producer-consumer model. The producer, operating on dedicated CPU nodes, fetches data from the distributed file system and preprocesses training data asynchronously. The consumer, i.e., the main training process on dedicated GPU nodes, receives the preprocessed data for training. The producer and consumer communicate through RPC calls, and use RDMA for lower latency if available.

The disaggregation enables flexible and elastic data preprocessing and provides opportunities for data reordering without incurring additional overhead. Leveraging this opportunity, we incorporate *data reordering* into data preprocessing to mitigate training

### Algorithm 1 Intra-batch reordering.

---

```

1: function INTRAREORDER( $\{d_1, \dots, d_n\}, m$ )
2:    $sorted\_samples \leftarrow \{d_1, \dots, d_n\}$ ,  $ret\_samples \leftarrow \emptyset$ ,
    $Groups \leftarrow \emptyset$ 
3:   Sort  $sorted\_samples$  in descending order based on  $d_i.size$ 
4:   for  $i = 1 \rightarrow m$  do
5:      $Group_i \leftarrow \emptyset$ ,  $Groups.append(Group_i)$ 
6:   for  $i = 1 \rightarrow n$  do
7:      $min\_index \leftarrow \argmin_j \sum_{d \in Group_j} d.size$ 
8:      $Groups[min\_index].append(sorted\_samples[i])$ 
9:   for  $i = 1 \rightarrow m$  do
10:     $ret\_samples.extend(Groups[i])$ 
11:  return  $ret\_samples$ 

```

---

stragglers caused by data heterogeneity. The reordering includes two levels: intra-microbatch and inter-microbatch. We emphasize that disaggregated data preprocessing ensures that the complex reordering does not interfere with the GPU training or impose extra overhead.

### 5.2 Intra-microbatch Reordering

**Insight.** To address intra-microbatch stragglers, we first identify the straggler by pinpointing the DP group with the largest training samples. As shown in Figure 6, the first DP group becomes a straggler as it contains the two largest training samples. Therefore, we reorder the training samples within the global batch by size. Specifically, as depicted in Figure 11, we reorder the training samples into the sequence  $[1, 3, 2, 4]$ , which distributes the computation more evenly.

**Intra-microbatch Reordering.** Leveraging this insight, we propose intra-microbatch reordering to balance the computational load within a global batch. Formally, the goal is to minimize the maximum computation time across data parallelism (DP) groups. This problem maps to the multiway number partitioning problem [38], which is NP-hard. Given the problem’s complexity and the large batch sizes common in production, finding an optimal solution is computationally infeasible. We therefore employ a lightweight, greedy partitioning algorithm that yields an approximation ratio of less than  $4/3$  [15]. Importantly, this reordering only occurs within the global batch, which only affects the sequence of gradient accumulation. Since gradient accumulation (i.e., summation) is a commutative operation, the intra-microbatch reordering preserves the training’s convergence semantics.

The detailed algorithm is summarized in Algorithm 1. The function *INTRAREORDER* receives the  $n$  original training samples and DP size  $m$ . This algorithm first sorts the training samples in descending order by the sample size (line 3). Then, it loops over the training samples and assigns the sample to the DP group with the current lowest computational load (line 6-8). It then returns the reordered samples (line 9-11). The algorithm has a time complexity of  $O(n \log n + m \times n)$ .

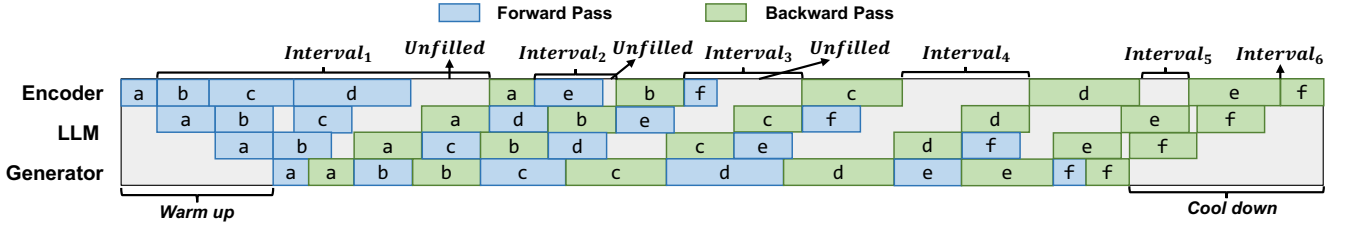


Figure 12: 1F1B pipeline scheme.

### 5.3 Inter-microbatch Reordering

As we discussed in §2.3, data heterogeneity leads to varied computation times across microbatches within the modality encoder and generator. The straggler microbatch prolongs the training by creating large pipeline bubbles. In the context of 1F1B pipeline scheme, the overall iteration time is primarily decided by pipeline bubbles and the computation time at the first PP stage of the modality encoder, as illustrated in Figure 12. Let the PP size be  $p$  and the number of microbatches be  $l$  ( $p = 4$  and  $l = 6$  in Figure 12). Typically,  $l$  is larger than  $p$  to reduce the proportion of time spent in the warm-up and cool-down phases. To help describe our solution, we first introduce a concept called pipeline intervals at the first PP stage. As shown in Figure 12, these intervals are typically filled with the forward pass, except for the last  $p - 1$  intervals (i.e.,  $interval_4$ ,  $interval_5$ , and  $interval_6$ ). Straggler microbatches in either the encoder or generator prolong these intervals or increase the unfilled area (i.e., bubble). Designing an efficient runtime algorithm to minimize pipeline bubbles presents a significant challenge.

**Insights.** We leverage two insights to solve this problem. The first insight involves minimizing the volume of intervals that are not filled. As shown in Figure 12, the last  $p - 1$  intervals (i.e.,  $interval_4$  to  $interval_6$ ) remain unfilled. These intervals become the pipeline bubbles and increase the training iteration. We observe a positive correlation between the volume of  $interval_i$  and the size of the  $i_{th}$  microbatch. The size refers to the computation time of the microbatch in modality encoder and generator. For instance,  $interval_4$  is significantly larger than  $interval_5$  and  $interval_6$  since the  $4_{th}$  microbatch (i.e., d) is the largest. By strategically reordering the training samples to position the smallest  $p - 1$  microbatches at the end, we are able to reduce these unfilled intervals' volume.

The second insight involves minimizing the unfilled area of the left intervals, which hides the training stragglers into the training pipeline as much as possible. The left intervals (i.e.,  $interval_1$  to  $interval_3$ ) are filled with the forward pass. As shown in Figure 12, the first interval is filled with by the  $2_{nd}$  to  $p_{th}$  forward passes. For subsequent intervals,  $interval_i$  is filled by the  $(i + p - 1)_{th}$  forward pass. However, the forward pass may not perfectly match the interval volumes, leading to unfilled areas. By evaluating the volume of  $interval_i$ , we place the microbatches, whose forward time most closely matches this volume, at the corresponding position, to minimize the unfilled area.

**Inter-microbatch Reordering.** Based on the preceding insights, we propose a runtime algorithm for inter-microbatch reordering designed to minimize pipeline bubbles. This algorithm is specifically developed for the 1F1B pipeline schedule. We will retrofit the

#### Algorithm 2 Inter-batch reordering.

```

1: function INTERREORDER( $\{m_1, \dots, m_l\}, p$ )
2:    $ret\_mb \leftarrow \emptyset$ ,  $mb \leftarrow \{m_1, \dots, m_l\}$ 
3:    $ret\_mb.append(MIN(mb))$ ,  $mb.remove(MIN(mb))$ 
4:    $rear\_mb \leftarrow SELECTMIN(mb, p - 1)$ ,  $mb.remove(rear\_mb)$ 
5:   for  $i = 1 \rightarrow l - p$  do
6:      $interval_i \leftarrow GETINTERVAL(ret\_mb, i)$ 
7:     if  $i == 1$  then
8:        $cur\_mb \leftarrow SELECTCLOSEST(mb, p - 1, interval_i)$ 
9:     else
10:       $cur\_mb \leftarrow SELECTCLOSEST(mb, 1, interval_i)$ 
11:       $ret\_mb.extend(cur\_mb)$ ,  $mb.remove(cur\_mb)$ 
12:    $ret\_mb.extend(rear\_mb)$ 
13:   return  $ret\_mb$ 

```

algorithm to VPP (i.e., interleaved 1F1B) later. Algorithm 2 summarizes the pseudo code. The *INTERREORDER* function accepts the initial microbatch sequence and the PP size  $p$ . The reordering process begins by scheduling the smallest microbatch first to ensure all pipeline stages are activated promptly (line 3). Subsequently, it reserves the  $p - 1$  smallest remaining microbatches for the end of the sequence to minimize unfilled intervals (lines 4 and line 12). The main loop (lines 5-11) then iterates through the remaining microbatches to fill the pipeline bubbles. In each loop iteration, it computes the interval size via the *GETINTERVAL* function and applies a heuristic to select microbatches that best fit this size. For the first interval, it greedily selects  $p - 1$  microbatches whose aggregate forward time most closely matches the interval size; for all subsequent bubbles, it selects the single best-fitting microbatch. This loop ensures maximal filling of the remaining intervals, which minimizes pipeline bubbles. Crucially, this reordering only alters the data sequence within the local batch of a single DP rank per training iteration. Therefore, analogous to intra-microbatch reordering, it only affects the sequence of gradient accumulation and consequently preserves the training's convergence semantics.

The functions *SELECTMIN* and *SELECTCLOSEST* operate with a time complexity of  $O(l)$ . The function *GETINTERVAL* calculates interval size using the current order  $ret\_mb$ . This calculation is facilitated by a dynamic programming algorithm that utilizes a recursive formula derived from pipeline dependencies. Specifically, the start time of each microbatch depends on two factors: the completion of the preceding microbatch in the same pipeline stage and the availability of input data from the upstream microbatch. Consequently, the end time of each microbatch is determined by the maximum of these two dependencies plus its own computation time. This



dynamic programming algorithm exhibits a complexity of  $O(p)$  per function invocation. The algorithm has a time complexity of  $O(l \times (l + p))$ .

Virtual pipeline parallelism (i.e., interleaved 1F1B) also follows the one forward and one backward pipeline scheme to reduce the memory footprint. The fundamental insights of our algorithm apply to any 1F1B-based pipeline, including VPP. We adapt the algorithm by computing multiple (i.e., VPP size) intervals and filling them with the corresponding number of forward passes from a single microbatch.

## 6 SYSTEM IMPLEMENTATION

We implement DistTrain with 6.3K lines of code in C++ and Python, and integrate it with Megatron-LM [57]. DistTrain handles failures by automatically recovering the training from the latest model checkpoint. To mitigate Tensor Parallelism (TP) overhead, we developed StepCCL, an in-house collective communication library. By leveraging the DMA engine for data transfers, StepCCL overlaps communication with computation, which frees the core Streaming Multiprocessors (SMs) to execute computation kernels (e.g., GEMM) without performance interference. A detailed description and evaluation are in §A.1.

**DistTrain manager and initializer.** DistTrain’s training manager, running on a dedicated CPU node, formulates the disaggregated model orchestration problem using Disciplined Convex Programming [31]. It employs the CVX solver [3] to solve this problem within a second. The manager records the optimal resource allocation and parallelism strategy to a configuration file, which the Kubernetes controller uses to launch the training task. As for the initializer, DistTrain uses PyTorch Distributed [9] library to initialize the communication groups. The initialization of each module in a multimodal LLM is implemented through parallelism units.

**Parallelism unit.** As discussed in §4.1, disaggregated model orchestration is implemented via a dedicated module: *parallelism unit*. During distributed training initialization, DistTrain first establishes communication groups within a parallelism unit. Each GPU process possesses a global and a local rank within its unit, facilitating distributed initialization. Subsequently, DistTrain initializes a communication broker to establish PP communication between adjacent parallelism units. All inter-unit communication traffic is routed via the communication broker. The communication broker is implemented by modifying Megatron-LM’s batched send/receive operations into discrete operations. It enables flexible communication between multiple upstream and downstream GPU processes by concentrating and scattering data as needed, while preserving data order. The communication broker employs a decentralized design, residing on the GPU of either the last PP stage in an upstream unit or the first PP stage in a downstream unit. To maximize communication bandwidth, the number of brokers between two units is determined by the greatest common divisor of their respective DP sizes. Consequently, the total inter-unit bandwidth scales effectively with the training workload, preventing the communication broker from becoming a training bottleneck. Moreover, Megatron-LM’s reliance on synchronous communication compels upstream stages to pause until downstream stages fully receive data, introducing unnecessary pipeline dependencies. To mitigate this, we implement

| Models     | # of Layers | Hidden Size | FFN Hidden Size | # of Heads | # of Groups |
|------------|-------------|-------------|-----------------|------------|-------------|
| Llama3-7B  | 32          | 4096        | 11008           | 32         | 32          |
| Llama3-13B | 40          | 5120        | 13824           | 40         | 40          |
| Llama3-70B | 80          | 8192        | 28672           | 64         | 8           |

Table 2: LLM backbone configurations.

asynchronous send operations, eliminating these superfluous dependencies, and redesign the communication topology to prevent potential deadlocks.

## 7 EVALUATION

In this section, we first use large-scale experiments to evaluate the overall performance improvements of DistTrain over Megatron-LM. Next, we use an ablation study to deep dive into DistTrain and show the effectiveness of each technique. Finally, we provide a case study to further evaluate DistTrain.

**Setup.** Our experiments are conducted on a production cluster, with each node equipped with 8 NVIDIA Ampere GPUs. GPUs within one node are interconnected by 300GB/s (bidirectional) NVLink, while nodes are connected by 4\*200 Gbps RDMA network based on RoCEv2 with rail-optimized topology. The overall experiments use up to 1296 GPUs, and the ablation study utilizes up to 96 GPUs. We use PyTorch 2.1.2 and NVIDIA CUDA 12.2. As for the modality, we focus on images and texts. We emphasize that DistTrain is also compatible with other modalities. Our evaluation focuses on images (i.e., vision). This choice is motivated by the prevalence of vision as a common non-textual modality in multimodal LLMs and the increasing prominence of capable vision-based MLLMs (e.g., Gemini [6] and Qwen2.5-VL [14]).

**Models.** For the LLM backbone, we choose the representative LLM architecture, Llama3 [7], which is widely used in both academia and industry. Table 2 lists the detailed model configurations. For encoder and generator, we use ViT-Huge [4] (0.63B) and SD 2.1 [11] (1B) respectively. The encoder and generator in our experiments are representative of state-of-the-art MLLM. The encoder (i.e., ViT-Huge) in our experiment aligns with the encoders of Qwen2.5-VL and Seed1.5-VL in terms of both model architecture and size. The generator (i.e., SD) is also adopted in many MLLMs (e.g., Step1X-Edit [44] and EMU [60]) with generation capabilities. The three LLM backbones (i.e., Llama3-7B, Llama3-13B, and Llama3-70B) are paired with ViT-Huge and SD to form multimodal LLMs denoted as MLLM-9B, MLLM-15B, and MLLM-72B. For the large multimodal LLM (i.e., MLLM-72B), we use high image resolution (i.e., 1024×1024) for generation since the large LLM is able to process more context information. For small models, we use low image resolution (i.e., 512×512).

**Datasets.** For our experiments, we use the representative open-source dataset, LAION-400M [55]. We generate training data by interleaving the image and text subsequences, forming input sequences up to 8192 tokens long. This dataset is also employed in our production multimodal LLM training. As detailed in §2.3, each training sample includes a varying number of image tokens and text tokens, which introduces data heterogeneity in multimodal LLM training.

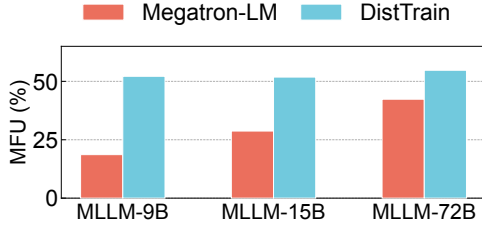


Figure 13: The overall MFU of DistTrain and Megatron-LM.

**Metrics.** We use Model FLOPs Utilization (MFU) as the primary metric. MFU measures the percentage of GPU FLOPs that are effectively utilized during training. We also use the training throughput to evaluate the training speed of DistTrain. Since DistTrain and Megatron-LM may utilize different numbers of GPUs due to varying model orchestration strategies, we also indicate the number of GPUs used in each experiment.

**Baselines.** We focus on training multimodal LLMs rather than traditional multimodal models like CLIP or LiT. DistMM [32] efficiently handles traditional multimodal training but lacks support for pipelines involving encoders, LLM backbones, and generators in multimodal LLMs. Multimodal LLMs combine multimodal models and LLMs to understand and generate high-quality multimodal data auto-regressively. Megatron-LM is selected as the primary baseline due to its established role as a highly optimized, open-source framework for LLM training that also provides supports for multimodal models. For ablation study of model orchestration, we integrate DistMM’s model orchestration strategy (resource allocation by model size and FLOPs) into DistTrain, naming this baseline DistMM\*. Note that DistMM does not support multimodal LLM training natively. DistMM\* only uses its orchestration strategy, with all other techniques from DistTrain.

## 7.1 Overall Performance

We first compare the overall performance of DistTrain against Megatron-LM on a large GPU cluster (up to 1296 GPUs). We retrofit Megatron-LM to support multimodal LLM training by integrating modality encoder and generator into the training pipeline. Megatron-LM employs monolithic model orchestration as described in §2.1. In Megatron-LM, we set the PP size of the LLM backbone to 1, 2, and 10 for Llama3-7B, Llama3-13B, and Llama3-70B, respectively. PP size is set to 1 for modality encoder and generator. We set TP size to 8, as each node consists of 8 GPUs connected by high-bandwidth NVLink. As for DistTrain, the parallelism strategy is determined by disaggregated model orchestration. In our experiments, one GPU is able to facilitate training ViT and SD. We replicate the modality encoder and generator across the GPUs within the TP group to process different images, whereas TP itself is not used. We set the global batch size to 1920.

The experimental results are shown in Figure 13 and Figure 14. Figure 13 shows the MFU. Figure 14 shows the training throughput. Due to the different model orchestration strategies of DistTrain and Megatron-LM, DistTrain uses 1056, 1216, and 1176 GPUs for MLLM-9B, MLLM-15B, and MLLM-72B, respectively. Megatron-LM uses 1296, 1280, and 1152 GPUs, respectively. Notably, within a total budget of 1296 GPUs, DistTrain intentionally allocates fewer

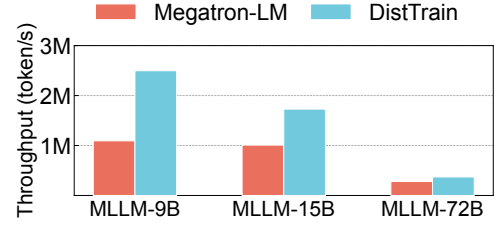


Figure 14: The overall throughput of DistTrain and Megatron-LM.

resources in some cases because adding more GPUs yields no further improvements in training throughput. This resource efficiency frees the remaining GPUs for concurrent tasks such as fine-tuning or inference. We summarize the findings as follows.

- As shown in Figure 13, DistTrain achieves 51.8%-54.7% MFU in large-scale multimodal LLM training. This performance closely approximates that of state-of-the-art unimodal (i.e., text) LLM training [35], which demonstrates the effectiveness of DistTrain in addressing the model and data heterogeneity in multimodal LLM training.
- DistTrain significantly outperforms Megatron-LM, delivering 1.7–2.8× the MFU and 1.7–2.2× the training throughput when training MLLM-9B and MLLM-15B with a similar number of GPUs. These performance gains largely stem from DistTrain’s disaggregated model orchestration. Megatron-LM’s monolithic strategy often leads to GPU underutilization, since it assigns too many GPUs to the modality encoder and generator. In contrast, DistTrain adaptively adjusts model orchestration based on specific model and data demands. Additionally, DistTrain’s disaggregated data preprocessing technique further improves efficiency.
- In the MLLM-72B training scenario, DistTrain outperforms Megatron-LM by 1.2× on MFU and 1.3× on training throughput with a similar number of GPUs. The high image resolution increases the execution time of the multimodal module, which introduces pipeline bubbles in LLM backbone. DistTrain addresses this by allocating additional GPUs to these modules to balance the pipeline. The disaggregated data preprocessing strategy continues to mitigate data heterogeneity, thereby increasing training efficiency.

## 7.2 Ablation Study

In this subsection, we perform microbenchmarks to evaluate the effectiveness of each DistTrain’s technique and utilize up to 96 GPUs. We reduce the cluster scale due to our limited budget. We set the global batch size of training to 128, 64, and 40 for MLLM-9B, MLLM-15B, and MLLM-72B, respectively.

**Disaggregated model orchestration.** We evaluate the MFU and training throughput using the model orchestration strategies of DistTrain, Megatron-LM, and DistMM\*. DistMM\* allocates GPUs according to the computation demands (flops) of each module. DistTrain uses 96, 80, and 82 GPUs for MLLM-9B, MLLM-15B, and MLLM-72B, respectively. Megatron-LM uses 96, 96, and 96 GPUs, while DistMM\* uses 88, 76, and 90 GPUs. The experimental results are shown in Figure 15. DistTrain consistently outperforms

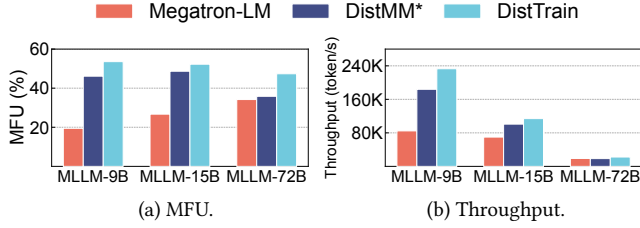


Figure 15: Disaggregated model orchestration.

| Model    | # of GPUs | Global Batch Size | Algorithm Overhead |
|----------|-----------|-------------------|--------------------|
| MLLM-72B | 1296      | 1920              | 922ms              |
| MLLM-72B | 648       | 960               | 641ms              |
| MLLM-72B | 324       | 480               | 441ms              |
| MLLM-72B | 112       | 240               | 133ms              |

Table 3: Overhead of disaggregated model orchestration.

the baseline strategies, achieving 1.3–2.7× higher MFU and 1.4–2.7× higher training throughput. Although DistMM\* outperforms Megatron-LM, it still lags behind DistTrain since it neglects the intricate performance model (§4.2) of multimodal LLM training. DistTrain’s disaggregated model orchestration optimally balances computational loads across the three modules and achieves high resource utilization. In production training, the running time of disaggregated model orchestration is less than one second, negligible compared to the days required for full training.

We also evaluate the running time of DistTrain’s disaggregated model orchestration to determine the optimal resource and parallelism configurations under different training settings, as detailed in Table 3. The algorithm completes in under one second. The overhead is negligible compared to the days or even weeks required for overall training.

**Disaggregated data preprocessing.** We evaluate the effectiveness of DistTrain’s disaggregated data preprocessing by comparing it against Megatron-LM’s data preprocessing, while keeping other components the same. Megatron-LM’s data preprocessing uses random ordering for training data. The effectiveness is gauged through metrics such as MFU and training throughput. We use the optimal resource allocation and parallelism strategy decided by DistTrain’s disaggregated model orchestration. Given that the model orchestration strategy remains unchanged, we do not indicate the number of GPUs. The experimental settings are the same as those in the experiment to evaluate disaggregated model orchestration. The results are shown in Figure 16. DistTrain consistently outperforms the baseline, achieving 1.03–1.11× higher MFU and throughput. The performance gap becomes more pronounced as the model size decreases. This is because the smaller model size leads to a higher data parallelism (DP) size, which causes more intra-microbatch heterogeneity. In essence, DistTrain’s two-level data reordering effectively mitigates data heterogeneity and enhances training efficiency. We do not measure the running time of the reordering algorithm as it operates on dedicated CPU nodes asynchronously. The data preprocessing overhead with and without disaggregation is evaluated in §7.3.

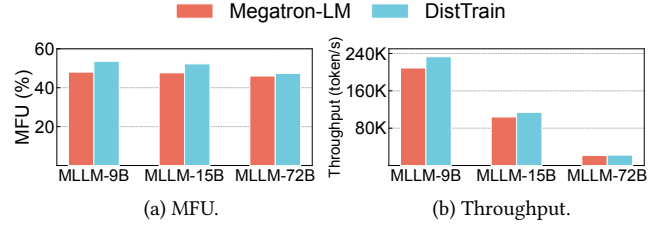


Figure 16: Disaggregated data preprocessing.

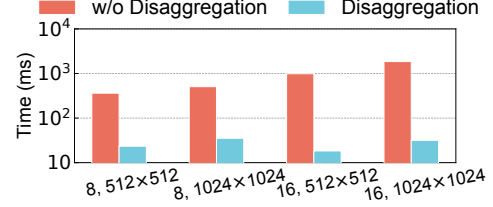


Figure 17: Overhead of data preprocessing.

### 7.3 Case Study

In this subsection, we first evaluate the overhead of data preprocessing. We then evaluate DistTrain under different frozen training settings.

**Overhead of data preprocessing.** We conduct an experiment to evaluate the overhead of data preprocessing, including decompression and reordering. Setting the DP size to one, we measure the average data preprocessing time per iteration on the GPU training side. We then compare data preprocessing time with and without disaggregated data preprocessing and use varying numbers of images and different image resolutions for one training iteration. The results, depicted in Figure 17, indicate disaggregating data preprocessing from training reduces preprocessing time from seconds to milliseconds. The first parameter in the x-axis represents the number of images, while the second parameter denotes the image resolution. In production training (§7.1), iteration times range from seconds to tens of seconds. Without disaggregation, preprocessing overhead is in seconds, significantly interferes with training. With disaggregated data preprocessing, the overhead reduces to milliseconds, which is negligible relative to total iteration time.

**Frozen training.** In real production training, some modules in multimodal LLM are frozen to stabilize training loss and enhance model effectiveness during different training phases [70]. We conduct a frozen training experiment under four specific training settings: complete module freezing (i.e., training projectors only), encoder-only training, LLM-only training, and generator-only training. In these scenarios, frozen modules neither compute weight gradients in backward pass nor update weights. The frozen modules still perform computations in forward pass. All other experimental setup aligns with those detailed in §7.2. DistTrain uses 96, 80, and 82 GPUs for MLLM-9B, MLLM-15B, and MLLM-72B under all three frozen settings. Megatron-LM uses 96, 96, and 96 GPUs for MLLM-9B, MLLM-15B, and MLLM-72B under all three frozen settings. The experimental results are in Figure 18 and Figure 19. DistTrain consistently outperforms Megatron-LM across all frozen training configurations, achieving 1.4–2.9× higher MFU and 1.2–2.9× higher training throughput. This pronounced performance gap

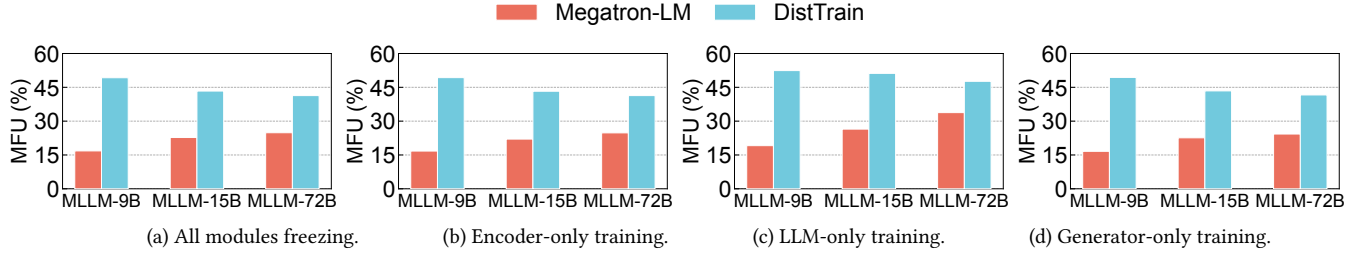


Figure 18: MFU under frozen training setting.

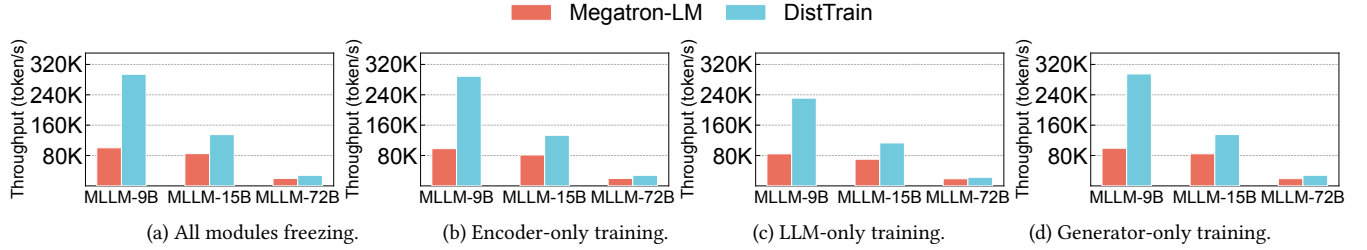


Figure 19: Throughput under frozen training setting.

underscores the challenges posed by Megatron-LM’s monolithic model orchestration in more complex training environments. In contrast, DistTrain adaptively adjusts model orchestration based on training settings and achieves high resource utilization.

## 8 DISCUSSION

**Parallelism strategies.** Many studies [34, 66, 78] optimize parallelism strategies to accelerate deep learning model training. However, they fall short of holistically considering data, pipeline, and tensor parallelism in multimodal LLM training, due to the large search spaces in large clusters. This limitation results in inefficient and sub-optimal parallelism strategy. Additionally, these methods generally assume homogeneity of training data—all training samples consume the same amount of computation. This assumption does not hold for multimodal LLMs where training samples of different modalities vary significantly in volume and computation demand. DistTrain leverages the specific training pattern of multimodal LLMs, and formulates a model orchestration problem for multimodal LLMs that integrates tensor, pipeline, and data parallelism simultaneously. Besides, DistTrain leverages disaggregated data reordering to address data heterogeneity.

**Sequence and expert parallelism.** Sequence parallelism (SP) [39] is designed to partition the training sequence into multiple subsequences for parallel training. It addresses the challenges of processing long sequences in LLMs. Expert parallelism (EP) [43], specifically devised for mixture-of-experts (MoE) LLMs [28], enables parallel training of multiple feedforward network (FFN) experts. These parallelism strategies are orthogonal to multimodal LLM training. In DistTrain, both SP and EP are integrated into the LLM backbone training.

**Heterogeneous hardware.** By disaggregating three modules (i.e., the modality encoder, LLM backbone, and generator) for multimodal LLM training, DistTrain supports using heterogeneous hardware for different modules to achieve various goals like improving

performance, reducing cost and increasing energy efficiency. This strategy assigns distinct computational workloads to the most appropriate hardware. For instance, ViT encoder is significantly less compute-intensive than LLM backbone. Consequently, we can place ViT encoder on more economical GPUs (e.g., NVIDIA L20). Such benefits are demonstrated in other LLM systems such as MegaScale-Infer [79] and Splitwise [47].

**Disaggregated data preprocessing.** The principal technique in DistTrain’s disaggregated data preprocessing is data reordering that addresses training stragglers caused by data heterogeneity. Although data reordering does not inherently necessitate disaggregation, we adopt disaggregation to mitigate the preprocessing overhead and improve system elasticity (e.g., allocating any arbitrary number of CPUs for preprocessing tasks). This approach is analogous to resource disaggregation in other systems scenarios, which aims to improve system elasticity and efficiency [56, 65].

## 9 RELATED WORK

**LLM training.** Many efforts have been made to optimize LLM training. For LLM pre-training, Megatron-LM [57] and DeepSpeed-Megatron [58] propose custom 3D-parallelism strategies. DeepSpeed-ZeRO [51] and Pytorch-FSDP [77] reduce redundant memory consumption in data parallelism. A set of works [8, 19, 20, 35, 67] overlap the communication and computation operators in LLM training. These systems optimizations of LLM training are orthogonal to DistTrain. They do not address model and data heterogeneity in multimodal LLM training. DistTrain integrates several of these optimizations in training the LLM backbone.

**Traditional multimodal training.** Traditional multimodal models (e.g., CLIP [48] and LiT [73]) have been widely studied in recent years. Many systems optimizations have been proposed to train such multimodal models efficiently. DistMM [32] optimizes multiple parallel encoders training for contrastive learning. Yet, these advancements primarily enhance traditional multimodal models



(e.g., CLIP [48] and LiT [73]), which do not integrate LLM for understanding and generation (e.g., Flamingo [13] and LLaVa [41]). They do not support encoder, LLM backbone and generator training pipeline in multimodal LLMs. Therefore, the state-of-the-art solution is to leverage Megatron-LM to train multimodal LLMs at large scale.

## 10 CONCLUSION

We present DistTrain, a disaggregated multimodal LLM training system. We identify the key challenges in training multimodal LLMs, i.e., model heterogeneity and data heterogeneity. DistTrain introduces disaggregated model orchestration to address model heterogeneity and disaggregated data preprocessing to address data heterogeneity. We evaluate DistTrain on a large production cluster with thousands of GPUs and show that it achieves 54.7% MFU and outperforms Megatron-LM by up to 2.2× on training throughput.

*This work does not raise any ethical issues.*

**Acknowledgments.** We sincerely thank the anonymous reviewers for their valuable feedback on this paper. Xin Jin is the corresponding author. Zili Zhang, Yinmin Zhong, and Xin Jin are also with the Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education.

This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4500700, the Scientific Research Innovation Capability Support Project for Young Faculty under Grant ZYGXQJNSKYCXNLZCXM-II, the Fundamental Research Funds for the Central Universities, Peking University, and the National Natural Science Foundation of China under Grant 62172008.

## REFERENCES

- [1] 2022. Techniques and Systems to Train and Serve Bigger Models. <https://icml.cc/virtual/2022/tutorial/18440>.
- [2] 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%\* chatgpt quality. <https://vicuna.lmsys.org/>.
- [3] 2024. CVXPY 1.5. <https://www.cvxpy.org/>.
- [4] 2024. Google ViT-Huge. <https://huggingface.co/google/vit-huge-patch14-224-in21k>.
- [5] 2024. Hello GPT-4o. <https://openai.com/index/hello-gpt-4o/>.
- [6] 2024. Introducing Gemini: our largest and most capable AI model. <https://blog.google/technology/ai/google-gemini-ai/>.
- [7] 2024. Meta Llama3. <https://llama.meta.com/>.
- [8] 2024. NVIDIA Transformer Engine. <https://github.com/NVIDIA/TransformerEngine>.
- [9] 2024. PyTorch Distributed Overview. [https://pytorch.org/tutorials/beginner/dist\\_overview.html](https://pytorch.org/tutorials/beginner/dist_overview.html).
- [10] 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115* (2024).
- [11] 2024. Stable Diffusion 2.1. <https://huggingface.co/stabilityai/stable-diffusion-2-1/>.
- [12] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [13] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. 2022. Flamingo: a visual language model for few-shot learning. *Advances in Neural Information Processing Systems* (2022).
- [14] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923* (2025).
- [15] Siddharth Barman and Sanath Kumar Krishnamurthy. 2020. Approximation algorithms for maximin fair division. *ACM Transactions on Economics and Computation (TEAC)* (2020).
- [16] Zalan Borsos, Raphaël Marinier, Damien Vincent, Eugene Kharitonov, Olivier Pietquin, Matt Sharifi, Dominik Roblek, Olivier Teboul, David Grangier, Marco Tagliasacchi, et al. 2023. Audioldm: a language modeling approach to audio generation. *IEEE/ACM transactions on audio, speech, and language processing* (2023).
- [17] Andy Brock, Soham De, Samuel L Smith, and Karen Simonyan. 2021. High-performance large-scale image recognition without normalization. In *International Conference on Machine Learning (ICML)*.
- [18] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*.
- [19] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Zheng, Yinmin Zhong, Xuanrun Zhang, Zuquan Song, Ziheng Jiang, Haibin Lin, Xin Jin, and Xin Liu. 2024. FLUX: Fast Software-based Communication Overlap On GPUs Through Kernel Fusion. *arXiv:2406.06858 [cs.LG]*.
- [20] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. 2024. Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning. In *ACM ASPLOS*.
- [21] Sanyuan Chen, Yu Wu, Chengyi Wang, Shujie Liu, Daniel Tompkins, Zhuo Chen, and Furu Wei. 2022. Beats: Audio pre-training with acoustic tokenizers. *arXiv preprint arXiv:2212.09058* (2022).
- [22] Xiaokang Chen, Zhiyu Wu, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, and Chong Ruan. 2025. Janus-pro: Unified multimodal understanding and generation with data and model scaling. *arXiv preprint arXiv:2501.17811* (2025).
- [23] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* (2023).
- [24] Chaorui Deng, Deyao Zhu, Kunchang Li, Chenhui Gou, Feng Li, Zeyu Wang, Shu Zhong, Weihao Yu, Xiaonan Nie, Ziang Song, et al. 2025. Emerging properties in unified multimodal pretraining. *arXiv preprint arXiv:2505.14683* (2025).
- [25] Runpei Dong, Chunrui Han, Yuang Peng, Zekun Qi, Zheng Ge, Jinrong Yang, Liang Zhao, Jianjian Sun, Hongyu Zhou, Haoran Wei, et al. 2023. Dreamllm: Synergistic multimodal comprehension and creation. *arXiv preprint arXiv:2309.11499* (2023).
- [26] Alexey Dosovitskiy. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [27] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. 2023. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378* (2023).
- [28] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. 2022. Glam: Efficient scaling of language models with mixture-of-experts. In *International Conference on Machine Learning (ICML)*.
- [29] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, et al. 2021. DAPPLE: A pipelined data parallel approach for training large models. In *ACM PPoPP*.
- [30] Michael Grant and Stephen Boyd. 2014. CVX: Matlab software for disciplined convex programming, version 2.1.
- [31] Michael Grant, Stephen Boyd, and Yinyu Ye. 2006. *Disciplined convex programming*.
- [32] Jun Huang, Zhen Zhang, Shuai Zheng, Feng Qin, and Yida Wang. 2024. DISTMM: Accelerating Distributed Multimodal Model Training. In *USENIX NSDI*.
- [33] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyoukjoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems*.
- [34] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond data and model parallelism for deep neural networks. *Conference on Machine Learning and Systems* (2019).
- [35] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibao Nong, et al. 2024. MegaScale: Scaling large language model training to more than 10,000 GPUs. In *USENIX NSDI*.
- [36] Diederik P Kingma, Max Welling, et al. 2013. Auto-encoding variational bayes.
- [37] Dan Kondratyuk, Lijun Yu, Xiuye Gu, José Lezama, Jonathan Huang, Grant Schindler, Rachel Hornung, Vignesh Birodkar, Jimmy Yan, Ming-Chang Chiu, et al. 2023. Videopoet: A large language model for zero-shot video generation. *arXiv preprint arXiv:2312.14125* (2023).
- [38] Richard Earl Korf. 2009. Multi-way number partitioning. In *Twenty-first international joint conference on artificial intelligence*.
- [39] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. 2021. Sequence parallelism: Long sequence training from system perspective. *arXiv preprint arXiv:2105.13120* (2021).

- [40] Haohe Liu, Zehua Chen, Yi Yuan, Xinhao Mei, Xubo Liu, Danilo Mandic, Wenwu Wang, and Mark D Plumbley. 2023. Audioldm: Text-to-audio generation with latent diffusion models. *arXiv preprint arXiv:2301.12503* (2023).
- [41] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2024. Visual instruction tuning. *Advances in Neural Information Processing Systems* (2024).
- [42] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. 2024. World model on million-length video and language with blockwise ringattention. *arXiv preprint arXiv:2402.08268* (2024).
- [43] Juncai Liu, Jessie Hui Wang, and Yimin Jiang. 2023. Janus: A unified distributed training framework for sparse mixture-of-experts models. In *ACM SIGCOMM*.
- [44] Shiyu Liu, Yucheng Han, Peng Xing, Fukun Yin, Rui Wang, Wei Cheng, Jiaqi Liao, Yingming Wang, Honghao Fu, Chunrui Han, et al. 2025. Step1x-edit: A practical framework for general image editing. *arXiv preprint arXiv:2504.17761* (2025).
- [45] Paulius Mikićevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2017. Mixed precision training. *arXiv preprint arXiv:1710.03740* (2017).
- [46] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *SC*.
- [47] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *ACM/IEEE ISCA*.
- [48] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*.
- [49] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. (2018).
- [50] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* (2019).
- [51] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *International Conference for High Performance Computing, Networking, Storage and Analysis*.
- [52] Saeed Rashidi, Matthew Denton, Srinivas Sridharan, Sudarshan Srinivasan, Amoghavarsha Suresh, Jade Nie, and Tushar Krishna. 2021. Enabling Compute-Communication Overlap in Distributed Deep Learning Training Platforms. In *ACM/IEEE ISCA*.
- [53] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *IEEE Conference on Computer Vision and Pattern Recognition*.
- [54] Paul K Rubenstein, Chulayuth Asawaroengchai, Duc Dung Nguyen, Ankur Bapna, Zalan Borsos, Félix de Chaumont Quitry, Peter Chen, Dalia El Badawy, Wei Han, Eugene Kharitonov, et al. 2023. Audiopalm: A large language model that can speak and listen. *arXiv preprint arXiv:2306.12925* (2023).
- [55] Christoph Schuhmann, Richard Vencu, Romain Beaumont, Robert Kaczmarczyk, Clayton Mullis, Aarush Katta, Theo Coombes, Jenia Jitsev, and Aran Komatsuzaki. 2021. Laion-400m: Open dataset of clip-filtered 400 million image-text pairs. *arXiv preprint arXiv:2111.02114* (2021).
- [56] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiyang Zhang. 2018. LegoOS: A disseminated, distributed OS for hardware resource disaggregation. In *USENIX OSDI*.
- [57] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [58] Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, et al. 2022. Using deepspeed and megatron to train megatron-turing nl 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990* (2022).
- [59] Quan Sun, Yuxin Fang, Ledell Wu, Xinlong Wang, and Yue Cao. 2023. Eva-clip: Improved training techniques for clip at scale. *arXiv preprint arXiv:2303.15389* (2023).
- [60] Quan Sun, Qiying Yu, Yufeng Cui, Fan Zhang, Xiaosong Zhang, Yuezhe Wang, Hongcheng Gao, Jingjing Liu, Tiejun Huang, and Xinlong Wang. 2023. Emu: Generative pretraining in multimodality. In *International Conference on Learning Representations (ICLR)*.
- [61] Chameleon Team. 2024. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818* (2024).
- [62] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [63] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*.
- [64] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature methods* (2020).
- [65] Chenxi Wang, Haoran Ma, Shi Liu, Yifan Qiao, Jonathan Eyolfson, Christian Navasas, Shan Lu, and Guoqing Harry Xu. 2022. MemLiner: Lining up Tracing and Application for a Far-Memory-Friendly Runtime. In *USENIX OSDI*.
- [66] Minjie Wang, Chien-chin Huang, and Jinyang Li. 2019. Supporting very large models using automatic dataflow graph partitioning. In *EuroSys*.
- [67] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, et al. 2022. Overlap Communication with Dependent Computation via Decomposition in Large Deep Learning Models. In *ACM ASPLOS*.
- [68] Wenhui Wang, Zhe Chen, Xiaokang Chen, Jiannan Wu, Xizhou Zhu, Gang Zeng, Ping Luo, Tong Lu, Jie Zhou, Yu Qiao, et al. 2024. Visionllm: Large language model is also an open-ended decoder for vision-centric tasks. In *Advances in Neural Information Processing Systems*.
- [69] Zhiyu Wu, Xiaokang Chen, Zizheng Pan, Xingchao Liu, Wen Liu, Damai Dai, Huazuo Gao, Yiyang Ma, Chengyue Wu, Bingxuan Wang, et al. 2024. DeepSeek-VL2: Mixture-of-Experts Vision-Language Models for Advanced Multimodal Understanding. *arXiv preprint arXiv:2412.10302* (2024).
- [70] Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. 2023. A survey on multimodal large language models. *arXiv preprint arXiv:2306.13549* (2023).
- [71] Lijun Yu, José Lezama, Nitesh B Gundavarapu, Luca Versari, Kihyuk Sohn, David Minnen, Yong Cheng, Vignesh Birodkar, Agrim Gupta, Xiuye Gu, et al. 2023. Language Model Beats Diffusion—Tokenizer is Key to Visual Generation. *arXiv preprint arXiv:2310.05737* (2023).
- [72] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. 2021. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* (2021).
- [73] Xiaohua Zhai, Xiao Wang, Basil Mustafa, Andreas Steiner, Daniel Keysers, Alexander Kolesnikov, and Lucas Beyer. 2022. Lit: Zero-shot transfer with locked-image text tuning. In *IEEE Conference on Computer Vision and Pattern Recognition*.
- [74] Ceng Zhang, Junxin Chen, Jiatong Li, Yanhong Peng, and Zebing Mao. 2023. Large language models for human-robot interaction: A review. *Biomimetic Intelligence and Robotics* (2023).
- [75] Duzhen Zhang, Yahan Yu, Chenxing Li, Jiahua Dong, Dan Su, Chenhui Chu, and Dong Yu. 2024. Mm-llms: Recent advances in multimodal large language models. *arXiv preprint arXiv:2401.13601* (2024).
- [76] Pan Zhang, Xiaoyi Dong Bin Wang, Yuhang Cao, Chao Xu, Linke Ouyang, Zhiyuan Zhao, Shuangrui Ding, Songyang Zhang, Haodong Duan, Hang Yan, et al. 2023. Internlm-xcomposer: A vision-language large model for advanced text-image comprehension and composition. *arXiv preprint arXiv:2309.15112* (2023).
- [77] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277* (2023).
- [78] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *USENIX OSDI*.
- [79] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, et al. 2025. MegaScale-Infer: Serving Mixture-of-Experts at Scale with Disaggregated Expert Parallelism. *arXiv preprint arXiv:2504.02263* (2025).

Appendices are supporting material that has not been peer-reviewed.

## A APPENDIX

### A.1 Mitigating TP overhead with StepCCL

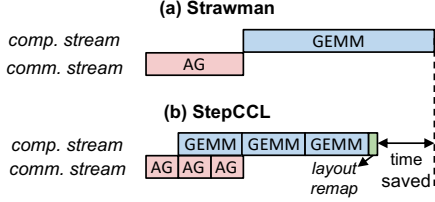


Figure 20: Overlapping communication and computation.

Tensor Parallelism (TP) is commonly adopted to facilitate training large Transformer-based models with multiple GPUs connected with high bandwidth (e.g., NVLinks). Specifically, TP divides the linear layers, into smaller sub-modules, which are then distributed across the GPUs. After parallel computation, all GPUs perform collective communication to aggregate data and produce identical results. The TP communication overhead severely degrades overall performance.

We implement the communication overlap with an in-house collective communication library called StepCCL to reduce the TP overhead. StepCCL is a PyTorch custom plugin that performs cross-GPU collective communication, including allgather, reduce-scatter, and allreduce, which is similar to NCCL. However, NCCL occupies several CUDA Streaming Multiprocessors (SMs) for executing its communication kernel and is known to harm the performance of its concurrent GEMM (matrix multiplication) [52]. To solve this, StepCCL leverages the DMA engine directly to perform data transmission without using any SM at all. This enables StepCCL and GEMM to run simultaneously on a GPU without slowing down each other. This cornerstone facilitates our subsequent development of communication overlap.

Figure 20 shows an example of how StepCCL works in overlapping the allgather (AG) operation with GEMM. We start by decomposing the single GEMM and the corresponding communication into several smaller pairs. Each small communication operation starts sequentially on a communication stream, with its paired GEMM executed on the default computation stream. The communication overhead is fully hidden except for the first allgather.<sup>1</sup> After all GEMMs finish, we perform an extra layout remapping operation (usually with negligible overhead) to ensure identical results with the baseline. Figure 21 describes the details of the layout remap process. In some rare cases during backward propagation, we find the remap overhead is high due to certain model dimensions. To mitigate this, we further overlap the remap with the computation of the weight gradients, so eventually we nearly get the full performance gain of the communication overlap.

Finally, although the overlap idea is also studied in many related works [8, 19, 67], we highlight the key differences of ours.

<sup>1</sup>If the number of allgather/GEMM is large enough, the only allgather in the critical path should have negligible overhead. But dividing a large GEMM into finer granularity sometimes could lead to overall slowdown. In practice, the number is actually configurable.

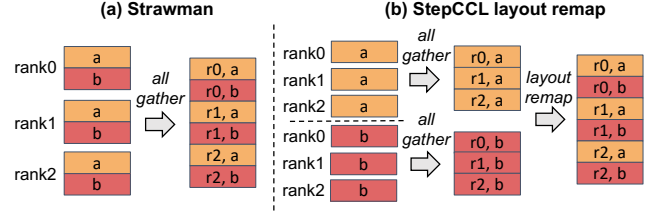


Figure 21: Layout remap.

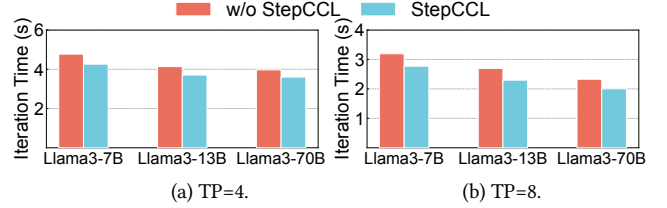


Figure 22: Overlapping the TP communication with computation.

Unlike prior work that fuses GEMM with TP communication into one CUDA kernel [8, 19], we choose a modular design and do not use fusion for more flexibility. For example, when TP communication is longer than GEMM, fusing them cannot fully hide the communication overhead. However, with the modular design, we are able to hide the communication with other modules without dependency (e.g., in cross-attention), which is not possible with the fused implementation. This enables broader adoption of StepCCL in many other scenarios.

**Evaluation.** To evaluate the effectiveness of StepCCL in mitigating the TP overhead, we conduct an experiment that measures the iteration time of the LLM backbone with training of one single PP stage (i.e., one minimal TP group) under various TP sizes. We compare the iteration time with and without StepCCL enabled. The results are shown in Figure 22. StepCCL significantly reduces the iteration time by overlapping the TP communication with computation. It outperforms the baseline by 1.1-1.12 $\times$  when the TP size is 4 and 1.15-1.17 $\times$  when the TP size is 8. The gains are more pronounced at large TP size, where communication overhead is more substantial. These findings confirm that StepCCL effectively mitigates TP overhead.